

Progress Report 3

Automated Product Inquiry Response System for Light Recycling Program



Student Name: Namesh Mathara Arachchi Vidanalage

Student ID: 300359798

Course: CSIS 4495 – Applied Research Project

Section: 002

Contents

Work Logs – Summarized	3
Summary	3
1. Research & Learning	3
2. Model Implementation	3
3. Reporting & Documentation	4
Work Logs – Detailed	4
Research & Learning	4
Techniques for Visual Similarity and Image Recognition	4
Why CNN?	5
CNN Models for Image Similarity	5
Why ResNet50?	7
Model Implementation	8
Repository Check-in Details	9
References	9

Work Logs – Summarized

Phase	Date	#	Number of Hours	Tr	Description of Work Done
Progress Report 3	17/03/2025	2			Researched on feature extraction and similarity methods for visual similarity and image recognition - Traditional Computer Vision Techniques
Progress Report 3	18/03/2025	2			Researched on feature extraction and similarity methods for visual similarity and image recognition - Deep Learning-Based Techniques
Progress Report 3	19/03/2025	2			Read more on Convolutional Neural Networks (CNNs), specifically ResNet50
Progress Report 3	20/03/2025	3			Updated the ml_model script to accommodate image similarity using ResNet50
Progress Report 3	21/03/2025	2			Modified the scripts and their requirements.txt to downgrade Python version to fix the compatibility issue - tensorflow and keras are not fully compatible with Python 3.12 yet
Progress Report 3	22/03/2025	1			Modified the scripts and their requirements.txt to downgrade Python version to fix the compatibility issue - tensorflow and keras are not fully compatible with Python 3.12 yet
Progress Report 3	23/03/2025	4			Modified the product_upload script to generate SAS image URL instead of public image URL when uploading data Drafted and finalized the progress report 3

Link to Work Logs Sheet - <https://docs.google.com/spreadsheets/d/1GEJrsX2NxDs6eYnWCq-M8xBMALygzySMiK9KGYYx6F8/edit?pli=1&gid=0#gid=0>

Summary

1. Research & Learning

- Researched feature extraction and similarity methods for visual similarity and image recognition (both **traditional Computer Vision techniques** and **Deep Learning-based techniques**).
- Studied **Convolutional Neural Networks (CNNs)**, including:
 - **ResNet50**
 - **VGG16 / VGG19**
 - **InceptionV3**
 - **DenseNet**
 - **EfficientNet**
 - **MobileNetV2**

[Read more in 'Work Logs - Detailed' section](#)

2. Model Implementation

- Updated the **ml_model** script to incorporate image similarity using **ResNet50**.
- Modified **scripts** and their **requirements.txt** to downgrade Python version to resolve compatibility issues with *TensorFlow* and *Keras* (which are not fully compatible with Python 3.12).

- Adjusted the **product_upload** script to generate *SAS image URLs* instead of *public image URLs* when uploading data.

[Read more in 'Work Logs - Detailed' section](#)

3. Reporting & Documentation

- Drafted and finalized **Progress Report 3** summarizing the key developments.

Work Logs – Detailed

Research & Learning

Techniques for Visual Similarity and Image Recognition

I explored the methods used in computer vision and machine learning to extract and compare features from images. These techniques focus on detecting key visual patterns like edges, textures, and shapes, which help in comparing images or identifying objects. Using deep learning models or traditional algorithms, these methods allow systems to recognize, match, or classify visual information effectively.

- **Deep Learning-Based Techniques**
 - Siamese Networks
 - Triplet Loss Networks
 - [Convolutional Neural Networks \(CNNs\)](#)
- **Traditional Computer Vision Techniques**
 - Histogram of Oriented Gradients (HOG)
 - Histogram Comparison
 - Traditional Feature Descriptors (SIFT, SURF, ORB)

Technique	Best For	Strengths	Weaknesses
Siamese Networks	Image matching, face verification, one-shot learning	Effective for comparing pairs, works with smaller datasets	Computationally expensive, prone to overfitting
CNN	Image classification, object detection, segmentation	Automatic feature learning, high accuracy	Requires large datasets, computationally intensive

Triplet Loss Networks	Face recognition, image retrieval	Good for fine-grained similarity learning	Requires careful triplet selection, computationally expensive
HOG	Pedestrian detection, object recognition	Simple, interpretable, good for human detection	Sensitive to scale and pose, less effective in cluttered environments
Histogram Comparison	Image retrieval based on color distribution	Simple, efficient, good for color-based retrieval	Sensitive to lighting, doesn't capture texture/shape
SIFT, SURF, ORB	Feature matching, image stitching, real-time applications	Robust to scale/rotation, ORB is fast	Expensive (SIFT/SURF), limited on complex tasks

Why CNN?

- **End-to-End Learning:** **CNNs** automatically learn hierarchical features from raw images, unlike **HOG**, **SIFT**, and **SURF**, which rely on handcrafted features.
- **Robustness:** **CNNs** handle scale, rotation, and illumination variations better than **HOG**, **SIFT**, and **SURF**, which struggle with large changes.
- **Large-Scale Data:** **CNNs** are more efficient for large datasets, while **SIFT**, **SURF**, and **ORB** are slower and less scalable.
- **Complex Features:** **CNNs** capture abstract features for better image similarity, whereas **HOG**, **Histogram Comparison**, and traditional descriptors are limited to simpler patterns.
- **Transfer Learning:** **CNNs** can leverage pre-trained models and fine-tuning, which is faster and more effective than training **Siamese** or **Triplet Loss Networks** from scratch.
- **Flexibility:** **CNNs** excel in general image similarity tasks, while **Siamese** and **Triplet Loss Networks** are specialized for specific use cases.
- **Scalability:** **CNNs** are more scalable, handling higher-resolution images and large datasets better than **SIFT**, **SURF**, and **ORB**.

CNN Models for Image Similarity

I researched pretrained CNNs like ResNet50, VGG16/VGG19, InceptionV3, DenseNet, EfficientNet, and MobileNetV2, which are used for image classification. They are trained on large datasets like ImageNet and can be fine-tuned for tasks like object detection or segmentation. Their designs vary, with features like skip connections (ResNet), deeper layers (VGG), and efficient scaling (EfficientNet, MobileNetV2).

- **Residual Networks:** ResNet introduces residual connections, allowing gradients to flow more easily through deep networks – **ResNet50**

- **Simple Deep Networks:** Uses a stack of 3x3 convolution layers followed by max-pooling layers, making the architecture easy to understand – **VGG16 / VGG19**
- **Inception Networks:** A key feature is the "Inception module" that performs multiple convolutions with different filter sizes in parallel, followed by concatenation - **InceptionV3**
- **Dense Networks:** Each layer receives input from all previous layers (dense connections), allowing the model to reuse features more efficiently - **DenseNet**
- **Efficient Networks:** EfficientNet uses a compound scaling method that scales depth, width, and resolution uniformly to achieve better efficiency - **EfficientNet**
- **Lightweight Networks for Mobile Devices:** MobileNets use depthwise separable convolutions to reduce the number of parameters and computation, making them efficient for mobile and embedded applications - **MobileNetV2**

Model	Advantages	Disadvantages	Best For	Example Use Case
ResNet50	• Strong performance with deep layers. • Efficient feature extraction with pre-trained weights. • Works well for transfer learning.	• More complex than shallow networks. • May not capture small-scale features as well as more specialized models.	• General image similarity • Transfer learning	• E-commerce product recommendation • Content-based image retrieval
VGG16 / VGG19	• Simple and easy to understand. • Strong transfer learning capability. • Works well for large datasets when fine-tuned.	• High computational cost. • Lack of advanced optimization techniques like residual connections.	• General image classification • Transfer learning	• Flower identification • Object classification
InceptionV3	• Efficient and scalable. • Multi-scale feature extraction using Inception modules. • Suitable for large datasets.	• More complex architecture. • Requires more memory and computational resources.	• Large-scale datasets • Multi-scale feature extraction	• Facial recognition • Product search
DenseNet	• Efficient parameter usage. • Improved gradient flow. • Strong feature reuse.	• Memory-intensive due to dense connections. • Slower training.	• Image classification • Medical image analysis	• Tumor detection • Image segmentation

EfficientNet	• Best trade-off between accuracy and efficiency. • Scalable across models.	• Complex to implement. • Might not always outperform simpler models in small datasets.	• General-purpose image recognition • Large-scale image tasks	• High-accuracy image classification • Image retrieval
MobileNetV2	• Lightweight and efficient. • Low latency for mobile and edge devices.	• Lower accuracy compared to larger models. • Limited to simpler tasks.	• Mobile applications • Real-time inference	• Augmented reality • Mobile product search

Why ResNet50?

- **Deep architecture with residual connections:** **ResNet50**'s deep structure and residual connections allow it to capture complex features and perform better at distinguishing subtle image differences, unlike **VGG16/VGG19**, which lacks residuals and struggles with deep networks.
- **Feature extraction efficiency:** **ResNet50** balances depth and computational cost effectively, whereas **InceptionV3** is more complex and requires more resources, and **EfficientNet** requires fine-tuning for similarity tasks.
- **Pretrained models:** **ResNet50**'s pretrained models, particularly on ImageNet, provide strong feature representations for image similarity tasks, unlike **DenseNet**, which is memory-intensive, and **MobileNetV2**, which sacrifices depth for efficiency.
- **Computational efficiency:** **ResNet50** offers a good trade-off between performance and cost, whereas **MobileNetV2** is optimized for mobile devices and **EfficientNet** is efficient but requires extra optimization for similarity tasks.

Model Implementation

Newly added **`find_best_match_image()`** method compares a new product image with existing product images to find the best match. It uses a pretrained ResNet50 model to extract features from both the new image and existing images. The images are processed and compared using cosine similarity, and the product with the highest similarity score is returned, along with the similarity percentage.

```

app.py 3 X
app.py > find_best_match_image
107
108 def find_best_match_image(new_product_image_url, products):
109
110     best_score = 0
111     best_match = None
112
113     # Preprocess new image
114     try:
115         response = requests.get(new_product_image_url, timeout=10)
116         img = Image.open(BytesIO(response.content)).convert('RGB')
117         img = img.resize((224, 224))
118         img_array = image.img_to_array(img)
119         img_array = np.expand_dims(img_array, axis=0)
120         img_array = preprocess_input(img_array)
121         new_features = model_img.predict(img_array)
122     except Exception as e:
123         print(f"Error processing new product image: {e}")
124         return None, 0
125
126     for product in products:
127         try:
128             response = requests.get(product['image_url'], timeout=10)
129             img = Image.open(BytesIO(response.content)).convert('RGB')
130             img = img.resize((224, 224))
131             img_array = image.img_to_array(img)
132             img_array = np.expand_dims(img_array, axis=0)
133             img_array = preprocess_input(img_array)
134             existing_features = model_img.predict(img_array)
135
136             similarity = cosine_similarity(new_features, existing_features)[0][0]
137
138             if similarity > best_score:
139                 best_score = similarity
140                 best_match = product
141
142         except Exception as e:
143             print(f"Error processing existing product image ({product.get('image_url')}): {e}")
144             continue
145
146     return best_match, best_score * 100 # Convert to percentage
147

```


Repository Check-in Details

The scripts I have checked into the GitHub repository since the Midterm Report are as follows:

- [front_end_script](#) / [back_end_script](#) – Modified the scripts and their requirements.txt to downgrade Python version.
- [product_upload_script](#) – Modified the script and its requirements.txt to downgrade Python version & and updated the script to generate SAS image URL instead of public image URL when uploading data.
- [ml_model_script](#) – Modified the script and its requirements.txt to downgrade Python version & updated the script to accommodate image similarity using ResNet50.
- [download_azure_table_script](#) – Created this new script to download data from existing Azure Tables.

Regular check-ins were maintained to ensure all progress was tracked in the repository.

References

- Understanding Image Similarity with Machine Learning:
<https://medium.com/@aleksej.gudkov/understanding-image-similarity-with-machine-learning-c8680c24dd56>
- High Similarity Image Recognition and Classification Algorithm Based on Convolutional Neural Network: <https://onlinelibrary.wiley.com/doi/10.1155/2022/2836486>
- Deep Learning Vs Traditional Computer Vision Techniques: Which Should You Choose?:
<https://medium.com/discover-computer-vision/deep-learning-vs-traditional-techniques-a-comparison-a590d66b63bd>
- Navigating the Intersection of Traditional Computer Vision and Deep Learning: A Guide to Building Production Systems: <https://www.strong.io/blog/navigating-the-intersection-of-traditional-computer-vision-and-deep-learning-a-guide-to-building-production-systems>
- An Introduction to Convolutional Neural Networks (CNNs):
<https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns>
- Convolutional Neural Networks (CNN) in Deep Learning:
<https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/>

- Exploring ResNet50: An In-Depth Look at the Model Architecture and Code Implementation:
<https://medium.com/@nitishkundu1993/exploring-resnet50-an-in-depth-look-at-the-model-architecture-and-code-implementation-d8d8fa67e46f>
- ChatGPT: OpenAI